Code Optimization & Performance Tuning Report

A Before/After Case Study in a C# Retail Sales Analytics Engine

`SalesAnalytics.Unoptimized` → `SalesAnalytics.Optimized`

Prepared for: Performance Optimization Task

Platform: .NET 8.0 (C# 12) · Environment: Linux container, 1 vCPU, Intel Xeon @ 2.10GHz

2026-09-07

1. Executive Summary

This report documents a complete performance-optimization exercise on a purpose-built C# console application that simulates a moderately complex, real-world workload: a retail sales analytics engine that ingests hundreds of thousands of transaction records and computes revenue aggregates, top-product rankings, regional customer counts, blocklist filtering, monthly trends, and a line-item export report.

Two functionally identical implementations were built side by side:

- SalesAnalytics.Unoptimized — a "before" version written the way real code often accretes: reasonable-looking loops and LINQ that individually seem fine, but that collectively re-scan the dataset many times, use O(n) lookup structures where O(1) ones belong, and allocate far more memory than necessary.
- SalesAnalytics.Optimized — an "after" version that fixes each identified inefficiency using idiomatic .NET techniques: single-pass aggregation, HashSet/Dictionary/PriorityQueue lookup structures, StringBuilder, pre-sized value-type (struct) records, and parallelized processing via Parallel.For with thread-local accumulation.

Both versions were run against the exact same generated dataset (fixed random seed) so the comparison isolates implementation choices rather than differences in input data. A 41-test automated correctness harness confirms the optimized engine produces byte-for-byte identical analytical output to the unoptimized engine — the refactor changed speed and memory, not behavior.

Headline results (n = 400,000 transaction records)

Metric	Before	After	Improvement
Core analytics pipeline (time)	4627.1 ms	149.8 ms	30.9x faster
Record loading (time)	58.3 ms	9.9 ms	5.9x faster
Export report, 7,500 rows (time)	1432.0 ms	8.1 ms	177.3x faster
Export report, 15,000 rows (time)	6696.1 ms	22.8 ms	293.9x faster
Export report, 15,000 rows (memory)	14660.78 MB	6.34 MB	2313.8x less
Core pipeline (peak working set)	142.13 MB	80.63 MB	-43%

The optimized engine was additionally validated at 4,000,000 records (10x the unoptimized benchmark size) to confirm it scales cleanly: the full analytics pipeline completed in 1272.5 ms with only 34.34 MB allocated and zero garbage-collection pauses — a scale the unoptimized algorithm cannot reach in practical time due to its $O(k \cdot n)$ and $O(n^2)$-leaning operations.

2. Test Application Overview

The sample application, “SalesAnalytics,” models a workload deliberately chosen to exercise the categories of bottleneck most commonly seen in production C# services: large in-memory collections, repeated aggregation over the same data, string-heavy report generation, and membership-testing against reference/lookup sets.

2.1 Domain model

Each transaction record carries: ProductId, CategoryId, RegionId, CustomerId, Quantity, UnitPrice, and a timestamp. A deterministic generator (fixed random seed) produces datasets of any size, along with a fixed “blocked product” reference set (simulating recalled/discontinued SKUs) that every transaction must be checked against.

2.2 Computations performed

- Total revenue grouped by product category
- Top-10 products ranked by revenue
- Count of distinct customers per sales region
- Count and total revenue of transactions whose product is not on the blocklist
- Monthly revenue trend across the dataset's ~24-month span
- A full CSV-style line-item export report (one line per transaction)

2.3 Solution layout

```
PerfDemo.sln
 src/
   SalesAnalytics.Core/          shared dataset generator + result/metric types
   SalesAnalytics.Unoptimized/   "before" engine (SalesRecord class + naive algorithms)
   SalesAnalytics.Optimized/     "after" engine (SalesRecord struct + optimized algorithms)
 tests/
   SalesAnalytics.Tests/         41 automated correctness checks, before vs. after
 reports/
   benchmarks.json                    raw measured metrics (this report's source data)
   unoptimized_console_output.txt     console transcript, before run
   optimized_console_output.txt       console transcript, after run
```

Both engine projects reference the same SalesAnalytics.Core project, guaranteeing they analyze identical input data — the only variable between runs is the implementation.

3. Methodology & Environment

3.1 Profiling approach

Rather than rely solely on a GUI profiler (not available in the target execution environment), the application embeds a lightweight BenchmarkHarness (SalesAnalytics.Core/BenchmarkHarness.cs) that captures the same categories of data Visual Studio's Diagnostic Tools / PerfView would report:

- Wall-clock elapsed time (System.Diagnostics.Stopwatch)
- Bytes allocated during the operation (GC.GetTotalAllocatedBytes)
- Garbage-collection counts per generation (GC.CollectionCount(0/1/2))
- Peak process working set (Process.PeakWorkingSet64)

Each measured run is preceded by a forced blocking GC.Collect() so every run starts from a clean, comparable baseline, and all results are written to reports/benchmarks.json — the report tables below are generated directly from that file, not hand-transcribed.

3.2 Fair-comparison controls

- Identical input data: both engines consume the same DataGenerator output for a given (size, seed) pair.
- Identical dataset size within each benchmark pair (400,000 records for the core pipeline; 7,500 and 15,000 for the export benchmark).
- Release build configuration (dotnet build -c Release) for all measured runs.
- Correctness parity verified independently (Section 6) so speed gains cannot be explained by "doing less work."

3.3 Environment

Item	Value
.NET SDK	8.0.130
Build configuration	Release
OS	Ubuntu 24.04 (Linux container)
CPU	Intel(R) Xeon(R) @ 2.10GHz, 1 logical processor available
Note on parallelism	Parallel.For in the optimized engine ran single-threaded in this environment; all measured gains below come from algorithmic/data-structure changes alone. On multi-core hardware the parallel pass would add further speedup.

4. Profiling Results: Identified Bottlenecks

Nine distinct inefficiencies were identified in the baseline implementation. Each is numbered in source-code comments (SalesAnalytics.Unoptimized/AnalyticsEngine.cs) so it can be traced directly to its fix in the optimized engine.

#	Location	Inefficiency	Complexity impact
1	BuildRecords	List<T> built with no capacity hint	Repeated internal array reallocation/copy as the list grows
2	BuildRecords	ProductName string eagerly formatted & stored on every record	n unnecessary string allocations (only ~10 are ever displayed)

#	Location	Inefficiency	Complexity impact
3	RevenueByCategory	Full LINQ Where+Sum re-scan once per category	O(categories × n) instead of O(n)
4	Top10ProductsByRevenue	Full re-scan once per distinct product	O(distinctProducts × n) instead of O(n)
5	Top10ProductsByRevenue	Full sort of every product just to take top 10	O(k log k) instead of O(k log 10)
6	DistinctCustomersByRegion	List<int>.Contains used to de-duplicate customer ids	O(n × seenSoFar) linear scan per record instead of O(1) HashSet lookup
7	ValidTransactions	List<int>.Contains against the blocklist	O(n × m) instead of O(n) with a HashSet
8	MonthlyRevenueTrend	“yyyy-MM” string key formatted per record	n short-lived string allocations instead of ~24
9	BuildLineItemExport	Report built with string += concatenation	$O(n^2)$ total character copies instead of O(n)

5. Optimizations Applied

Each fix below corresponds to the numbered bottleneck in Section 4. Representative before/after code excerpts are shown; full source is included in the accompanying package.

5.1 Value-type records instead of eagerly-allocated strings (#1, #2)

The domain model moved from a reference-type class with an eagerly-formatted display string to a lean readonly record struct holding only ids, stored in a pre-sized array.

```
// BEFORE — class + eager string allocation, unsized List<T>
public class SalesRecord {
    public string ProductName { get; set; }   // "Product-" + id, built for EVERY record
    ...
}
var list = new List<SalesRecord>();            // grows/reallocates as it fills
foreach (...) list.Add(new SalesRecord { ProductName = "Product-" + id, ... });

// AFTER — struct, no display string, exact-size array
public readonly record struct SalesRecord(
    int ProductId, int CategoryId, int RegionId,
    int CustomerId, int Quantity, decimal UnitPrice, long TimestampTicks);
var array = new SalesRecord[_raw.Count];        // one allocation, exact size
for (var i = 0; i < _raw.Count; i++) array[i] = new SalesRecord(...);
```

5.2 Single-pass aggregation instead of one scan per key (#3, #4)

Rather than re-scanning the dataset once per category and once per product, the optimized engine computes every metric — category revenue, product revenue, regional customer sets, blocklist totals, and monthly trend — in one parallelized O(n) pass, using a thread-local accumulator merged at the end.

```
// BEFORE — one full scan PER category, one full scan PER product
foreach (var categoryName in categoryNames)
    result[categoryName] = records.Where(r => r.CategoryName == categoryName).Sum(r =>
r.LineTotal);
foreach (var productId in distinctProductIds)
    revenueByProduct.Add((productId, records.Where(r => r.ProductId == productId).Sum(r =>
r.LineTotal)));

// AFTER — one pass computes everything at once (shown simplified; the real
// version uses Parallel.For with a thread-local accumulator, see 5.5)
foreach (var r in records) {
    revenueByCategory[r.CategoryId] += r.LineTotal;
    revenueByProduct[r.ProductId]    = revenueByProduct.GetValueOrDefault(r.ProductId) +
r.LineTotal;
    customersByRegion[r.RegionId].Add(r.CustomerId);
    if (!blocked.Contains(r.ProductId)) { validCount++; validRevenue += r.LineTotal; }
}
```

5.3 Bounded min-heap for Top-10 instead of a full sort (#5)

```
// BEFORE — sorts every one of the ~1,500 products just to keep 10
revenueByProduct.Sort((a, b) => b.Revenue.CompareTo(a.Revenue));
return revenueByProduct.Take(10).ToList();

// AFTER — PriorityQueue<int, decimal> min-heap capped at 10 elements
if (heap.Count < 10) heap.Enqueue(productId, revenue);
else {
    heap.TryPeek(out _, out var minRevenueInHeap);
    if (revenue > minRevenueInHeap) { heap.Dequeue(); heap.Enqueue(productId, revenue); }
}
```

5.4 HashSet lookups instead of List.Contains (#6, #7)

```
// BEFORE — O(n) scan on every membership check
var seen = new List<int>();
if (!seen.Contains(customerId)) seen.Add(customerId);
var blocked = raw.BlockedProductIds.ToList();
if (blocked.Contains(r.ProductId)) continue;

// AFTER — O(1) average-case lookup
var seen = new HashSet<int>();
seen.Add(customerId);                       // Add() is a no-op if already present
var blocked = new HashSet<int>(raw.BlockedProductIds);
if (blocked.Contains(r.ProductId)) continue;
```

5.5 Parallelized aggregation (best practice: async/parallel programming)

The single combined pass described in 5.2 is executed with Parallel.For using the thread-local accumulator + final-merge pattern, so independent per-record work is spread across available cores and only the (cheap) merge step needs synchronization:

```
Parallel.For(0, records.Length,
    localInit: () => new PartitionState(),
    body: (i, state, local) => { /* accumulate record i into 'local' */ return local; },
    localFinally: local => { lock (mergeLock) { /* merge 'local' into shared totals
*/ } });
```

In this report's 1-vCPU test environment this yields no additional wall-clock benefit (there is only one core to schedule onto), so every measured speedup in Section 6 is attributable purely to the algorithmic and

data-structure changes above. On multi-core production hardware this pass would scale further with available cores.

5.6 Integer keys instead of per-record string formatting (#8)

```
// BEFORE — formats a "yyyy-MM" string for every single record
var key = $"{r.Timestamp.Year}-{r.Timestamp.Month:D2}";

// AFTER — cheap integer key; the ~24 distinct human-readable labels are
// formatted once each, only when producing final output
var monthKey = dt.Year * 100 + dt.Month;
...
monthlyTrend[$"{year}-{month:D2}"] = revenue;   // executed ~24 times total, not n times
```

5.7 StringBuilder instead of string concatenation (#9)

```
// BEFORE — O(n^2): every += allocates a new string and copies everything so far
var report = "TransactionExport\n";
foreach (var r in records)
    report += $"{r.Timestamp:yyyy-MM-dd},{r.ProductName},...\n";

// AFTER — O(n): one growable buffer, capacity pre-estimated
var sb = new StringBuilder(records.Length * 90 + 32);
sb.Append("TransactionExport\n");
foreach (var r in records)
    sb.Append(dt.ToString("yyyy-MM-
dd")).Append(',').Append("Product-").Append(r.ProductId)...Append('\n');
return sb.ToString();
```

6. Performance Results

All figures below are taken directly from reports/benchmarks.json, produced by real, timed runs of both compiled Release binaries against identical input data. Full console transcripts are included in the package (unoptimized_console_output.txt / optimized_console_output.txt).

6.1 Core analytics pipeline — 400,000 records

Computes revenue by category, top-10 products, distinct customers per region, blocklist filtering, and monthly trend, in a single run.

Metric	Unoptimized	Optimized	Change
Elapsed time	4627.1 ms	149.8 ms	30.9x faster
Bytes allocated	16.03 MB	4.84 MB	3.3x less
Gen0 / Gen1 / Gen2 GCs	0/0/0	0/0/0	no GC pressure
Peak working set	142.13 MB	80.63 MB	43% lower

6.2 Record loading (BuildRecords) — 400,000 records

Metric	Unoptimized	Optimized	Change
Elapsed time	58.3 ms	9.9 ms	5.9x faster
Bytes allocated	69.62 MB	18.31 MB	3.8x less

6.3 Export / report generation — scaling behavior

This benchmark isolates the string-concatenation fix (#9). Because the unoptimized version is genuinely $O(n^2)$, it is run at a smaller, clearly-labeled scale (7,500 / 15,000 rows) — large enough to demonstrate the effect, small enough to finish in reasonable time. The key finding is not just the absolute speedup but the growth curve: doubling the row count roughly quadruples the unoptimized runtime, while the optimized version grows linearly.

Rows	Before (ms)	After (ms)	Speedup	Before (MB)	After (MB)
7,500	1432	8.1	177.3x	3665	3.17
15,000	6696	22.8	293.9x	14661	6.34
Growth, 2x rows	4.7x time	2.8x time	-	-	-

The unoptimized export also drove 196 generation-0, 196 generation-1, and 196 full generation-2 garbage collections at 15,000 rows — the growing intermediate strings were large enough to be promoted onto the Large Object Heap, forcing expensive full collections. The StringBuilder-based version triggered zero collections at the same size.

6.4 Scale validation — 4,000,000 records (optimized engine only)

To confirm the optimized engine scales past the size used for the head-to-head comparison, it was additionally run at 4,000,000 records — 10x the size used above, and a size at which the unoptimized algorithm's O(distinctProducts × n) and O(n × seenSoFar) operations would not complete in practical time.

Metric	400,000 records	4,000,000 records
Record loading time	9.9 ms	632.4 ms
Full pipeline time	149.8 ms	1272.5 ms
Full pipeline allocation	4.84 MB	34.34 MB
GC pauses	0	0

7. Correctness Verification

A speedup is only meaningful if both versions produce the same answers. SalesAnalytics.Tests runs both engines against five identical datasets (sizes 0, 1, 500, 5,000, and 20,000 records, spanning multiple random seeds and edge cases) and asserts, per scenario:

- Record counts match
- Revenue-by-category totals match exactly
- Distinct-customer-per-region counts match exactly
- Monthly revenue trend matches exactly, including the set of distinct months
- Valid (non-blocklisted) transaction count and revenue match exactly
- Top-10 products match as a set, with matching revenue amounts and matching descending order
- The full line-item export text is byte-for-byte identical between engines

A network-restricted environment prevented restoring the usual xUnit/NUnit NuGet packages, so a small dependency-free test harness (TestHarness.cs, ~50 lines) was written to provide the same practical guarantee: run every check, print pass/fail, and return a non-zero exit code on any failure so it still integrates with CI. Swapping in a standard framework later is a drop-in change — see the comment at the top of TestHarness.cs.

Result: 41 / 41 tests passed. The optimized engine is behaviorally identical to the unoptimized engine across every scenario tested.

8. Conclusions & Recommendations

8.1 Summary of findings

The dominant cost in the unoptimized engine was algorithmic — re-scanning the full dataset once per category and once per product, and using linear-scan lookups (List.Contains) where hashed lookups belong. These are Big-O problems that no amount of micro-tuning fixes; they required restructuring the aggregation into a single pass. The secondary cost was allocation pressure — an eagerly-built display string per record, and O(n^2) string concatenation for the export report — which showed up directly as extra garbage-collection cycles.

8.2 Recommended practices going forward

- Default to a single aggregation pass over a collection; if a value is needed once per unique key, accumulate into a Dictionary during one scan instead of re-filtering per key.
- Reach for HashSet<T>/Dictionary<TKey,TValue> for any membership test or de-duplication over more than a handful of items; List<T>.Contains is O(n) and easy to miss in code review.
- Use PriorityQueue<TElement,TPriority> (or an equivalent bounded heap) for top-N/bottom-N problems instead of sorting the full collection.

- Use StringBuilder for any text built incrementally in a loop; a single += on a string inside a loop is an $O(n^2)$ red flag.
- Prefer value types (readonly record struct) for small, high-volume, short-lived data to reduce heap allocation and GC pressure, and avoid pre-computing derived strings that are rarely used.
- For independent per-item work over large collections, Parallel.For with a thread-local accumulator scales cleanly across cores with minimal synchronization overhead.

8.3 Suggested next steps

- Re-run this benchmark suite on multi-core production hardware to quantify the additional gain from the Parallel.For pass (not observable in this single-vCPU test environment).
- Wire SalesAnalytics.Tests into CI, or restore network access and swap in xUnit for richer test reporting/coverage tooling.
- If dataset sizes grow well beyond memory (tens of millions+ of records), consider streaming aggregation (IAsyncEnumerable / batched reads) rather than materializing the full record array.

Appendix A: Reproducing These Results

```
# from the solution root
dotnet build -c Release

dotnet run --project src/SalesAnalytics.Unoptimized -c Release --no-build --
reports/benchmarks.json
dotnet run --project src/SalesAnalytics.Optimized   -c Release --no-build --
reports/benchmarks.json

dotnet run --project tests/SalesAnalytics.Tests -c Release
```

Each program prints a human-readable summary to the console and appends a structured JSON entry to the given path (default reports/benchmarks.json), so repeated runs accumulate a comparable history rather than overwriting prior data.

Appendix B: Package Contents

- PerfDemo.sln — solution file
- src/SalesAnalytics.Core — shared data generator, result types, benchmark harness
- src/SalesAnalytics.Unoptimized — baseline (“before”) engine and console runner
- src/SalesAnalytics.Optimized — refactored (“after”) engine and console runner
- tests/SalesAnalytics.Tests — 41-case correctness harness
- reports/benchmarks.json — raw measured metrics (source of every number in this report)
- reports/unoptimized_console_output.txt, reports/optimized_console_output.txt — full console transcripts
- PERFORMANCE_REPORT.md — Markdown version of this report